Programming 2

Tutorial 10

Exercise 1 (beginner)

Create a file named `ArrayListOperations.java` using `ArrayList` and perform the following tasks:

- Add an integer to the `ArrayList`
- Add a floating-point number to the `ArrayList`
- Add a boolean value to the `ArrayList`
- Add a string to the `ArrayList`
- Print out the 4 values from the `ArrayList` to the screen

Exercise 2 (beginner)

Create file `ArrayListIntegerLoop.java` using `ArrayList<>` and perform the following tasks:

- Use a generic `ArrayList` of type `Integer` (`ArrayList<Integer> myArray = new ArrayList<Integer>();`)
- Use a loop to input numbers from `1` to `10` into the `myArray` list.
- Use a loop to display numbers from `1` to `10` from the `myArray` array.

Exercise 3 (beginner-intermediate)

In this exercise, your objective is to design and implement a Java class named `Product`. The `Product` class will contain attributes such as `name` and `price`. `Product` should implement the `Serializable` interface in Java to enable your class instances to be converted into a byte stream, which can be transmitted over a network or stored externally without losing the object's structure.

Complete the following tasks:

- Define a comprehensive table of domain constraints applicable to the attributes of the `Product` class.
- Design and implement the `Product` class within the `tut11.product` package.

Exercise 4 (intermediate)

In this exercise, your objective is to implement a Java class named `DAO` (Data Access Object) abstract class with CRUD (Create, Read, Update, Delete) methods.

- The `DAO` class should be defined as `abstract` and `generic`.
- The `DAO` class should contain a list of objects of type `T` to store data.
- The `DAO` class should have an `add` method to add an object of type `T` to the list.
- The `DAO` class should have a `remove` method to remove an object from the list. Ensure consistency with how objects are identified in other methods.
- The `DAO` class should have **two abstract methods** `update` and `find`. You need to implement these methods in specific concrete subclasses. The `update` method is used to update information of an object already in the list, while the `find` method must retrieve objects using an integer ID. Consider how the type parameter `T` can support this requirement.
- You are provided with an interface that allows objects to expose an integer ID:

```
public interface Identifiable {
    int getId();
}
```

- The `DAO` class should have a `getList` method to return the list of objects of type `T`.

Notes:

- ✓ Students need to pay attention to the use of generics to ensure flexibility and reusability of the `DAO` class.
- ✓ The `DAO` class must remain generic and reusable for different entity types.
- ✓ The `DAO` class must be type-safe and should not rely on type casting.
- ✓ Students should handle and manage exceptions properly in the methods of the `DAO` class.

Exercise 5 (intermediate)

In this exercise, you create a class `ProductDAO` inheriting from the `DAO` class (in Exercise 4) and write code to implement the abstract methods.

- The `ProductDAO` class should extend the `DAO` class with a type parameter of `Product`, indicating that it operates on objects of type `Product`.
- The `update` method should iterate through the list of products and update the product with the same ID as the provided object.

- The `find` method should iterate through the list of products and return the product with matching ID. If no product is found, it should return `null`.
- Write a test program to validate the correctness of the implemented functionality.

Note:

- ✓ Ensure that the methods are implemented correctly to work with the `Product` class and handle any edge cases appropriately.